

COMPLETE_TROUBLESHOOTING_GUIDE

HelixCode Troubleshooting Guide

Overview

This guide provides comprehensive troubleshooting information for common HelixCode issues, including startup problems, runtime errors, performance issues, and configuration problems. Each section includes symptoms, causes, and step-by-step resolution procedures.

Quick Start Issues

Server Won't Start

Symptoms: - Server fails to start with fatal error - Port binding errors - Configuration loading failures

Common Causes & Solutions:

Configuration File Missing

```
# Error: Failed to load configuration
ls -la config/config.yaml
# If missing, copy from example
cp config/config.yaml.example config/config.yaml
```

Invalid Configuration Syntax

```
# Check YAML syntax
yamllint config/config.yaml

# Validate configuration
./bin/helixcode --validate-config
```

Port Already in Use

```
# Check what's using the port
lsof -i :8080

# Kill process using port
kill -9 <PID>

# Or change port in config
vim config/config.yaml
# server:
#   port: 8081
```

Database Connection Failure

```
# Check PostgreSQL is running
systemctl status postgresql

# Test connection
psql -h localhost -U helix -d helixcode_prod

# Check environment variables
echo $HELIX_DATABASE_PASSWORD
```

Redis Connection Failure

```
# Check Redis is running
systemctl status redis

# Test connection
redis-cli ping

# Check environment variables
echo $HELIX_REDIS_PASSWORD
```

CLI Commands Fail

Symptoms: - Command not found errors - Permission denied - Invalid arguments

Solutions:

Binary Not in PATH

```
# Add to PATH
export PATH=$PATH:/path/to/helixcode/bin

# Or use full path
/path/to/helixcode/bin/helixcode --help
```

Permission Issues

```
# Make executable
chmod +x bin/helixcode

# Check file permissions
ls -la bin/helixcode
```

Invalid Command Syntax

```
# Get help
./bin/helixcode --help
```

```
# Command-specific help
./bin/helixcode task --help
```

Database Issues

Connection Refused

Symptoms: - “connection refused” errors - Database initialization failures - Migration errors

Troubleshooting Steps:

1. Check Database Status

```
# PostgreSQL service status
systemctl status postgresql
```

```
# Check if port is open
netstat -tlnp | grep 5432
```

1. Verify Connection Parameters

```
# Check environment variables
echo $HELIX_DATABASE_PASSWORD
```

```
# Test connection manually
psql -h localhost -p 5432 -U helix -d helixcode_prod
```

1. Check Database Logs

```
# PostgreSQL logs
tail -f /var/log/postgresql/postgresql-*.log
```

```
# Docker logs
docker logs helixcode-postgres
```

1. Network Issues

```
# Test network connectivity
telnet localhost 5432
```

```
# Check firewall rules
iptables -L | grep 5432
```

Schema Initialization Failures

Symptoms: - “failed to initialize schema” warnings - Missing tables or indexes - Foreign key constraint errors

Resolution:

1. Manual Schema Creation

```
-- Connect to database
psql -U helix -d helixcode_prod
```

```
-- Run schema creation
\i postgres-init.sql
```

1. Check Database Permissions

```
-- Grant necessary permissions
GRANT ALL PRIVILEGES ON DATABASE helixcode_prod TO helix;
GRANT ALL ON SCHEMA public TO helix;
```

1. Reset Database

```
# Drop and recreate database
dropdb helixcode_prod
createdb helixcode_prod
```

Connection Pool Exhaustion

Symptoms: - “too many connections” errors - Slow database response times - Application hangs

Solutions:

1. Increase Connection Pool Size

```
database:
  max_connections: 50
  min_connections: 5
  connection_lifetime: "1h"
```

1. Monitor Connection Usage

```
-- Check active connections
SELECT count(*) FROM pg_stat_activity WHERE datname =
    'helixcode_prod';

-- Check connection age
SELECT pid, username, client_addr, backend_start, query_start
FROM pg_stat_activity
WHERE datname = 'helixcode_prod';
```

1. Optimize Connection Settings

```
database:
  max_idle_connections: 10
  connection_timeout: "30s"
```

Redis Issues

Connection Failures

Symptoms: - Redis connection errors - Session storage failures - Cache misses

Troubleshooting:

1. Check Redis Service

```
# Service status
systemctl status redis
```

```
# Check if running
redis-cli ping
```

1. Verify Configuration

```
# Check Redis config
cat /etc/redis/redis.conf | grep -E "(bind|port|requirepass)"
```

```
# Test authentication
redis-cli -a $HELIX_REDIS_PASSWORD ping
```

1. Network Connectivity

```
# Test connection
telnet localhost 6379
```

```
# Check Redis logs
tail -f /var/log/redis/redis.log
```

Memory Issues

Symptoms: - Redis memory full errors - Eviction warnings - Performance degradation

Solutions:

1. Increase Memory Limit

```
maxmemory 1gb
maxmemory-policy allkeys-lru
```

1. Monitor Memory Usage

```
# Check memory usage
redis-cli info memory
```

```
# Monitor key count
redis-cli dbsize
```

1. Configure Persistence

```
save 900 1
save 300 10
save 60 10000
```

Worker Connection Issues

SSH Connection Failures

Symptoms: - Worker addition fails - “connection refused” errors - Authentication failures

Troubleshooting Steps:

1. Verify SSH Configuration

```
# Test SSH connection manually
ssh -i ~/.ssh/id_rsa user@worker-host

# Check SSH key permissions
ls -la ~/.ssh/id_rsa
chmod 600 ~/.ssh/id_rsa
```

1. Check Network Connectivity

```
# Test basic connectivity
ping worker-host

# Check SSH port
nmap -p 22 worker-host
```

1. Worker Host Requirements

```
# Check if Go is installed on worker
ssh user@worker-host "go version"

# Check available disk space
ssh user@worker-host "df -h"
```

1. Firewall Configuration

```
# Check firewall rules
iptables -L | grep ssh

# SELinux issues (if applicable)
setsebool -P ssh_chroot_rw_homedirs 1
```

Worker Health Check Failures

Symptoms: - Workers marked as unhealthy - Task assignment failures - Worker pool exhaustion

Resolution:

1. Check Worker Logs

```
# On worker host
tail -f /var/log/helixcode/worker.log
```

1. Verify Worker Configuration

```
# Check worker registration
./bin/helixcode worker list

# Test worker health
curl http://worker-host:2222/health
```

1. Resource Monitoring

```
# Check system resources on worker
ssh user@worker-host "top -b -n1 | head -20"

# Check disk usage
ssh user@worker-host "df -h"
```

LLM Provider Issues

API Key Problems

Symptoms: - Authentication failures - Rate limit errors - Model not found errors

Solutions:

1. Verify API Keys

```
# Check environment variables
echo $OPENAI_API_KEY
echo $ANTHROPIC_API_KEY
echo $GEMINI_API_KEY

# Test API key validity
curl -H "Authorization: Bearer $OPENAI_API_KEY" \
  https://api.openai.com/v1/models
```

1. Check Rate Limits

```
# Monitor API usage
./bin/helixcode llm status

# Check rate limit headers in responses
```

```
curl -v https://api.openai.com/v1/chat/completions \
-H "Authorization: Bearer $OPENAI_API_KEY"
```

1. Provider-Specific Issues

OpenAI

```
curl https://status.openai.com/api/v1/status
```

Anthropic

```
curl https://status.anthropic.com/
```

Google AI

```
curl https://status.cloud.google.com/
```

Model Availability Issues

Symptoms: - “model not found” errors - Invalid model name errors - Provider switching failures

Resolution:

1. Check Available Models

List available models

```
./bin/helixcode llm models
```

Test model access

```
./bin/helixcode llm test --model gpt-4
```

1. Update Model Configuration

```
llm:
  providers:
    openai:
      models:
        - gpt-4
        - gpt-3.5-turbo
    anthropic:
      models:
        - claude-3-sonnet
        - claude-3-haiku
```

1. Provider Fallback Configuration

```
llm:
  fallback_providers:
    - openai
    - anthropic
    - gemini
  retry_attempts: 3
  retry_delay: "1s"
```


Performance Issues

High CPU Usage

Symptoms: - CPU utilization > 80% - Slow response times - System becomes unresponsive

Diagnosis & Resolution:

1. Profile CPU Usage

```
# Start CPU profiling
go tool pprof http://localhost:8080/debug/pprof/profile
```

```
# Analyze profile
(pprof) top10
(pprof) web
```

1. Common CPU Issues

- Inefficient algorithms ($O(n^2)$ complexity)
- Memory allocation pressure
- Lock contention
- Goroutine leaks

1. Optimization Steps

```
# Run performance optimization
./bin/helixcode performance optimize

# Check goroutine count
curl http://localhost:8080/debug/pprof/goroutine | head -20
```

High Memory Usage

Symptoms: - Memory usage growing continuously - Frequent garbage collection - Out of memory errors

Troubleshooting:

1. Memory Profiling

```
# Get memory profile
go tool pprof http://localhost:8080/debug/pprof/heap
```

```
# Analyze allocations
(pprof) top10
(pprof) web
```

1. GC Tuning

```
performance:
garbage_collection:
```

```
enabled: true
GOGC: 100
GOMAXPROCS: 4
target_memory_usage: 1073741824 # 1GB
```

1. Memory Leak Detection

```
# Check for goroutine leaks
curl http://localhost:8080/debug/pprof/goroutine

# Monitor heap growth
watch -n 5 'curl -s http://localhost:8080/debug/vars |
jq .memstats.HeapAlloc'
```

Slow Response Times

Symptoms: - API response times > 200ms - P95/P99 latency high - User-facing delays

Performance Analysis:

1. Latency Profiling

```
# Profile with latency
go tool pprof -http=:8081 http://localhost:8080/debug/pprof/
profile
```

1. Database Query Optimization

```
-- Check slow queries
SELECT query, calls, total_time, mean_time
FROM pg_stat_statements
ORDER BY mean_time DESC
LIMIT 10;

-- Add indexes for slow queries
CREATE INDEX CONCURRENTLY idx_tasks_status ON tasks(status);
```

1. Cache Optimization

```
cache:
  size: 10000
  ttl: "1h"
  hit_rate_target: 0.95
```

Security Issues

Authentication Failures

Symptoms: - Login failures - Invalid token errors - Session expiration issues

Troubleshooting:

1. JWT Configuration

```
# Check JWT secret
echo $HELIX_AUTH_JWT_SECRET | wc -c # Should be > 32

# Verify token format
./bin/helixcode auth validate-token <token>
```

1. Session Management

```
# Check Redis session storage
redis-cli keys "session:*"

# Verify session expiry
redis-cli ttl "session:<session_id>"
```

1. Password Hashing

```
auth:
  bcrypt_cost: 12 # Increase for better security
  session_expiry: 604800 # 7 days
```

Authorization Issues

Symptoms: - Access denied errors - Permission failures - Role-based access problems

Resolution:

1. Check User Roles

```
# Verify user permissions
./bin/helixcode user info <user_id>

# Check role assignments
./bin/helixcode user roles <user_id>
```

1. RBAC Configuration

```
auth:
  roles:
    admin:
      - "read:*"
      - "write:*"
      - "delete:*"
    developer:
      - "read:projects"
      - "write:projects"
      - "read:tasks"
```

```
user:
  - "read:own"
```

Security Scan Failures

Symptoms: - Security scan errors - SonarQube connection failures - Snyk authentication issues

Troubleshooting:

1. SonarQube Issues

```
# Check SonarQube status
```

```
curl http://localhost:9000/api/system/status
```

```
# Verify project configuration
```

```
curl http://localhost:9000/api/projects/search?projects=helixcode
```

1. Snyk Issues

```
# Check Snyk token
```

```
echo $SNYK_TOKEN
```

```
# Test Snyk authentication
```

```
snyk auth $SNYK_TOKEN
```

1. Security Scan Configuration

```
security:
  scanning:
    enabled: true
    sonar_host: "http://sonarqube:9000"
    snyk_token: "${SNYK_TOKEN}"
    zero_tolerance: true
```

Networking Issues

Port Binding Failures

Symptoms: - “address already in use” errors - Port binding failures - Service unreachable

Solutions:

1. Check Port Usage

```
# Find process using port
```

```
lsof -i :8080
```

```
# Kill conflicting process
```

```
kill -9 <PID>
```

1. Change Port Configuration

```
server:
  address: "0.0.0.0"
  port: 8081 # Change from default 8080
```

1. Firewall Configuration

```
# Open port in firewall
```

```
ufw allow 8080
```

```
# Check SELinux
```

```
semanage port -a -t http_port_t -p tcp 8080
```

TLS/SSL Issues

Symptoms: - Certificate validation errors - HTTPS connection failures - Mixed content warnings

Resolution:

1. Certificate Validation

```
# Check certificate
```

```
openssl x509 -in cert.pem -text -noout
```

```
# Test SSL connection
```

```
openssl s_client -connect localhost:443
```

1. TLS Configuration

```
server:
```

```
  tls:
```

```
    enabled: true
```

```
    cert_file: "/etc/ssl/certs/helixcode.crt"
```

```
    key_file: "/etc/ssl/private/helixcode.key"
```

```
    min_version: "1.2"
```

1. Certificate Renewal

```
# Renew Let's Encrypt certificate
```

```
certbot renew
```

```
# Restart services
```

```
systemctl restart helixcode
```

Docker Deployment Issues

Container Startup Failures

Symptoms: - Container exits immediately - Health check failures - Service unavailable

Troubleshooting:

1. Check Container Logs

View container logs

```
docker logs helixcode-server
```

Follow logs in real-time

```
docker logs -f helixcode-server
```

1. Container Health Checks

Check container health

```
docker ps
```

Inspect container

```
docker inspect helixcode-server
```

1. Environment Variables

Check environment in container

```
docker exec helixcode-server env
```

Verify required variables are set

```
docker exec helixcode-server echo $HELIX_DATABASE_PASSWORD
```

Docker Compose Issues

Symptoms: - Service dependencies not starting - Network connectivity issues - Volume mount failures

Resolution:

1. Service Dependencies

Check service status

```
docker-compose ps
```

Start specific service

```
docker-compose up -d postgres
```

Check service logs

```
docker-compose logs postgres
```

1. Network Issues

Check network connectivity

```
docker-compose exec helixcode-server ping postgres
```

```
# Verify network configuration
docker network ls
```

1. Volume Mounts

```
# Check volume permissions
ls -la /var/lib/docker/volumes/

# Fix permissions
sudo chown -R 1001:1001 /var/lib/docker/volumes/helixcode_data
```

Monitoring & Alerting Issues

Metrics Collection Failures

Symptoms: - Missing metrics data - Prometheus scraping failures - Grafana dashboard empty

Troubleshooting:

1. Prometheus Configuration

```
scrape_configs:
  - job_name: 'helixcode'
    static_configs:
      - targets: ['localhost:8080']
    metrics_path: '/metrics'
```

1. Metrics Endpoint

```
# Test metrics endpoint
curl http://localhost:8080/metrics

# Check response format
curl -s http://localhost:8080/metrics | head -20
```

1. Grafana Configuration

```
# Check Grafana data sources
curl -u admin:admin http://localhost:3000/api/datasources

# Verify dashboard configuration
curl -u admin:admin http://localhost:3000/api/dashboards
```

Alert Configuration Issues

Symptoms: - Alerts not triggering - False positive alerts - Missing alert notifications

Resolution:

1. Alert Rules

```
alerts:
  - name: high_cpu
    condition: "cpu_usage > 80"
    severity: critical
    channels: ["slack", "email"]
    cooldown: "5m"
```

1. Notification Channels

```
notifications:
  slack:
    webhook_url: "${HELIX_SLACK_WEBHOOK_URL}"
  email:
    smtp_server: "smtp.gmail.com"
    smtp_port: 587
    username: "${HELIX_EMAIL_USERNAME}"
    password: "${HELIX_EMAIL_PASSWORD}"
```

1. Test Alerting

```
# Test alert manually
./bin/helixcode alert test --name high_cpu

# Check alert logs
tail -f /var/log/helixcode/alerts.log
```

Log Analysis

Log Level Configuration

Symptoms: - Missing debug information - Too verbose logging - Log rotation issues

Configuration:

```
logging:
  level: "info" # debug, info, warn, error
  format: "json" # json, text
  output: "stdout" # stdout, file
  file:
    path: "/var/log/helixcode/server.log"
    max_size: "100MB"
    max_age: "30d"
    max_backups: 10
```

Log Analysis Tools

```
# Search for errors
grep "ERROR" /var/log/helixcode/*.log

# Count error types
```



```
grep "ERROR" /var/log/helixcode/*.log | cut -d' ' -f4 | sort |  
    uniq -c | sort -nr
```

```
# Analyze response times  
grep "response_time" /var/log/helixcode/*.log |  
    jq .response_time | sort -n
```

Backup & Recovery Issues

Backup Failures

Symptoms: - Backup jobs failing - Incomplete backups - Storage space issues

Troubleshooting:

1. Backup Configuration

```
backup:  
  enabled: true  
  schedule: "0 2 * * *" # Daily at 2 AM  
  retention: "30d"  
  compression: true  
  encryption: true  
  destinations:  
    - type: "s3"  
      bucket: "helixcode-backups"  
      region: "us-east-1"
```

1. Storage Issues

```
# Check disk space  
df -h /backup  
  
# Test backup destination  
aws s3 ls s3://helixcode-backups/
```

1. Backup Verification

```
# List backups  
./bin/helixcode backup list  
  
# Verify backup integrity  
./bin/helixcode backup verify <backup_id>
```

Recovery Issues

Symptoms: - Recovery process fails - Data corruption - Inconsistent state

Resolution:

1. Recovery Testing

```
# Test recovery procedure
./bin/helixcode backup restore --dry-run <backup_id>
```

```
# Perform actual recovery
./bin/helixcode backup restore <backup_id>
```

1. Data Consistency

```
# Check database integrity
psql -d helixcode_prod -c "SELECT count(*) FROM tasks;"
```

```
# Verify application state
curl http://localhost:8080/health
```

Emergency Procedures

System Unresponsive

Immediate Actions:

1. Check System Resources

```
# CPU usage
top -b -n1 | head -20
```

```
# Memory usage
free -h
```

```
# Disk usage
df -h
```

1. Restart Services

```
# Graceful restart
systemctl restart helixcode
```

```
# Force restart if needed
systemctl stop helixcode
systemctl start helixcode
```

1. Database Recovery

```
# Check database status
systemctl status postgresql
```

```
# Restart if needed
systemctl restart postgresql
```

Data Loss Scenarios

Recovery Steps:

1. Assess Damage

```
# Check database connectivity
psql -d helixcode_prod -c "SELECT 1;"

# Verify data integrity
./bin/helixcode db check
```

1. Restore from Backup

```
# List available backups
./bin/helixcode backup list

# Restore latest backup
./bin/helixcode backup restore --latest
```

1. Verify Recovery

```
# Check application functionality
curl http://localhost:8080/health

# Verify data consistency
./bin/helixcode db validate
```

Getting Help

Support Resources

1. Documentation

- [API Reference](#)
- [Deployment Guide](#)
- [Security Guide](#)
- [Performance Tuning Guide](#)

2. Community Support

- GitHub Issues: <https://github.com/helixcode/helixcode/issues>
- Discussion Forums: <https://github.com/helixcode/helixcode/discussions>

3. Professional Support

- Enterprise Support: support@helixcode.dev
- Emergency Hotline: +1-800-HELIX-01

Diagnostic Information

System Information Collection:

```
# Collect diagnostic information
./bin/helixcode diagnostics collect
```

```
# Generate support bundle
```

```
./bin/helixcode support bundle --output helixcode-support.tar.gz
```

Support Bundle Contents: - System information - Configuration files - Log files - Performance metrics - Database schema - Network configuration

Prevention Best Practices

Proactive Monitoring

1. Set Up Alerts

```
alerts:
```

- name: disk_space_low
condition: "disk_usage_percent > 85"
severity: warning
- name: memory_high
condition: "memory_usage_percent > 90"
severity: critical

1. Regular Health Checks

```
# Automated health monitoring
```

```
*/5 * * * * /path/to/helixcode/bin/helixcode health check >> /  
var/log/helixcode/health.log
```

1. Performance Baselines

```
# Establish performance baselines
```

```
./bin/helixcode performance baseline create
```

```
# Monitor against baselines
```

```
./bin/helixcode performance baseline compare
```

Regular Maintenance

1. Update Schedule

```
# Weekly updates
```

```
0 2 * * 1 /path/to/scripts/update-helixcode.sh
```

```
# Security patches
```

```
0 3 * * * /path/to/scripts/security-update.sh
```

1. Backup Verification

```
# Daily backup verification
```

```
0 4 * * * /path/to/helixcode/bin/helixcode backup verify --latest
```

1. Log Rotation

```
logging:
  rotation:
    enabled: true
    max_size: "100MB"
    max_age: "30d"
    compression: true
```

This troubleshooting guide covers the most common HelixCode issues and provides systematic approaches to diagnosis and resolution. For issues not covered here, please check the GitHub repository or contact support. docs/COMPLETE_TROUBLESHOOTING_GUIDE.md ## Sources verified Sources verified 2026-05-29: <https://go.dev/doc/devel/release> (Go 1.26.3 confirmed latest stable, 2026-05-07) ; <https://www.postgresql.org/support/versioning/> (PostgreSQL 15 supported, latest minor 15.18, EOL 2027-11-11) ; <https://github.com/ollama/ollama/blob/main/docs/api.md> (Ollama default endpoint <http://localhost:11434> confirmed) ; <https://github.com/redis/redis/releases> (Redis 7+ requirement valid; latest GA 8.8.0) — all version/endpoint references for Go, PostgreSQL, Redis, and Ollama in this troubleshooting guide cross-checked against official sources and found current; project version authority is go.mod (inner module go 1.26) + CLAUDE.md §3.1. Negative finding: OpenAI's hosted API-reference page (platform.openai.com) returned HTTP 403 to automated fetch and could not be machine-verified this run.